

Mini-projet 03 — Sauvegarder les résultats (JSON & CSV)

Pourquoi ce mini-projet ?

Ton outil produit des résultats (des cibles, des ports ouverts...). Si tu les gardes seulement dans une variable Python, ils **disparaissent** dès que le script se termine. Il faut les **écrire sur le disque** dans un format standard, pour les relire plus tard ou les donner à un autre outil (Excel, un rapport, un collègue). Les deux formats rois : **JSON** (structures imbriquées) et **CSV** (tableau plat).

1. Sérialiser, ça veut dire quoi ?

« Sérialiser » = transformer un objet Python (liste, dict...) en **texte** qu'on peut écrire dans un fichier. « Désérialiser » = l'inverse (relire le texte et reconstruire l'objet Python). JSON et CSV sont deux façons de faire ça.

2. JSON — pour les structures

JSON ressemble énormément aux dicts/listes Python. Le module `json` fait la conversion dans les deux sens. Retiens **4 fonctions**, et la différence tient à une lettre : le **s** = « string ».

Fonction	Fait quoi
<code>json.dump(obj, f)</code>	écrit <code>obj</code> dans un fichier <code>f</code>
<code>json.dumps(obj)</code>	renvoie <code>obj</code> en chaîne (s = string) — pas de fichier
<code>json.load(f)</code>	lit un fichier <code>f</code> et reconstruit l'objet Python
<code>json.loads(s)</code>	lit une chaîne <code>s</code> et reconstruit l'objet

Écrire dans un fichier JSON

```
import json

cibles = [{"IP": "10.0.0.5", "PORT": 22}, {"IP": "10.0.0.5", "PORT": 80}]

with open("resultats.json", "w", encoding="utf-8") as f:
    json.dump(cibles, f, indent=2, ensure_ascii=False)
```

- `"w"` → mode **écriture** (write). `"r"` = lecture (défaut).
- `indent=2` → indente joliment (lisible à l'œil). Sans ça, tout est sur une ligne.
- `ensure_ascii=False` → garde les accents (é, à...) au lieu de les transformer en `é`.

Le fichier obtenu :

```
[
  {
    "IP": "10.0.0.5",
    "PORT": 22
  },
  {
    "IP": "10.0.0.5",
    "PORT": 80
  }
]
```

Relire un fichier JSON

```
with open("resultats.json", encoding="utf-8") as f:
    donnees = json.load(f)
# donnees est une VRAIE liste de dicts Python, prête à l'emploi
print(donnees[0]["IP"]) # 10.0.0.5
```

Pièges JSON à connaître :

- JSON ne connaît **ni les tuple ni les set**. Un tuple est écrit comme une liste ; un `set` provoque une **erreur** → convertis-le en

liste avant (`list(mon_set)`).

- Les **clés** d'un dict deviennent **toujours des chaînes** en JSON. `{1: "a"}` se relit `{"1": "a"}`.
- JSON ne sait pas sérialiser un **objet de ta classe** directement (on verra comment au Module B).

3. CSV — pour les tableaux (façon Excel)

CSV = « Comma-Separated Values » : un tableau plat, une ligne = une entrée, les colonnes séparées par des virgules. Parfait pour ouvrir dans Excel/LibreOffice. Pour une liste de dicts, le plus simple est `DictWriter` / `DictReader`.

Écrire un CSV

```
import csv

cibles = [{"IP": "10.0.0.5", "PORT": 22}, {"IP": "10.0.0.5", "PORT": 80}]

with open("resultats.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=["IP", "PORT"])
    writer.writeheader()      # écrit la ligne d'en-tête : IP,PORT
    writer.writerows(cibles)  # écrit toutes les lignes d'un coup
```

⚠ **Piège Windows (important pour toi)** : à l'ouverture d'un CSV en écriture, mets **toujours** `newline=""`. Sans ça, sous Windows tu obtiens une **ligne vide entre chaque ligne** de données. C'est LE piège CSV numéro 1.

Le fichier obtenu :

```
IP,PORT
10.0.0.5,22
10.0.0.5,80
```

Relire un CSV

```
with open("resultats.csv", newline="", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    lignes = [ligne for ligne in reader]  # liste de dicts
```

Piège CSV n°2 — tout est du texte. Un CSV ne connaît que des **chaînes**. En relisant, `PORT` vaudra `"22"` (str), pas `22` (int). Si tu veux l'int, reconvertis avec `int(...)` — exactement comme au mini-projet 01.

4. JSON ou CSV : lequel choisir ?

Situation	Format
Données imbriquées (dict dans dict, listes de listes)	JSON
Tableau plat à ouvrir dans Excel / trier / filtrer	CSV
Échanger avec une API / un autre programme	JSON
Donner à un humain non-technique	CSV

🎯 Ta mission

Objectif : repartir de ta liste de cibles (comme au MP01) et écrire 3 fonctions de sauvegarde/chargement.

Tu peux réutiliser ton `parse_cibles` du MP01, ou juste partir d'une liste écrite en dur pour aller vite. Écris ensuite :

1. `sauver_json(cibles, chemin)` → écrit la liste dans un fichier JSON (indenté, accents gardés).
2. `charger_json(chemin)` → relit le fichier et **renvoie** la liste de dicts.
3. `sauver_csv(cibles, chemin)` → écrit la liste dans un CSV (avec en-tête, et `newline=""` !).

Test de bout en bout (le « round-trip ») :

```
cibles = [{"IP": "10.0.0.5", "PORT": 22}, {"IP": "10.0.0.5", "PORT": 80}]
```

```
sauver_json(cibles, "resultats.json")
recharge = charger_json("resultats.json")
print(recharge == cibles)      # doit afficher : True

sauver_csv(cibles, "resultats.csv")
# puis ouvre resultats.csv à la main pour vérifier qu'il n'y a PAS de lignes vides
```

Bonus :

- `charger_csv(chemin)` qui relit le CSV ET reconvertit `PORT` en `int`.
- fais en sorte que `charger_csv(...) == cibles` renvoie aussi `True` (attention au piège « tout est str »).

✓ Critères de réussite

- `charger_json(...) == cibles` renvoie `True` (aller-retour parfait).
- Le fichier `resultats.json` est lisible (indenté), accents intacts.
- `resultats.csv` s'ouvre proprement, sans ligne vide entre les données.
- Chaque fonction ouvre son fichier avec `with`.

Si tu bloques — indices progressifs

Étape	Indice
Import	En haut du fichier : <code>import json</code> et <code>import csv</code> .
Écrire le JSON	Ouvre en <code>"w"</code> , puis <code>json.dump(cibles, f, indent=2, ensure_ascii=False)</code> .
Relire le JSON	Ouvre en lecture, <code>return json.load(f)</code> . N'oublie pas le <code>return</code> !
Écrire le CSV	<code>DictWriter</code> a besoin de <code>fieldnames</code> = les clés. Puis <code>writeheader()</code> et <code>writerows(cibles)</code> .
Bonus reconversion	Après <code>DictReader</code> , refais un dict par ligne avec <code>int(ligne["PORT"])</code> — une compréhension du MP01 fera l'affaire.

Où ranger : `mini-projets/03-sauvegarde-json-csv/sauvegarde.py`.

Astuce Git : tu peux ajouter `resultats.json` et `resultats.csv` à ton `.gitignore` si tu ne veux pas les versionner (ce sont des fichiers générés). On en reparlera.

Quand c'est prêt : `git add .` → `commit` → `push`, puis « c'est poussé ».